

키와 기본 명령어

Redis UI 접속

 **docker desktop** PERSONAL

      G 

Containers [Give feedback](#)

Container CPU usage 
0.42% / 1600% (16 CPUs available)

Container memory usage 
312.9MB / 14.87GB

[Show charts](#)

  Only running

☐	Name	Container ID	Image	Port(s)	Actions
☐ >	 postgres	-	-	-	  
☐ v	 redis	-	-	-	  
☐	 stack	fb3ccc190b6c	 redis/redis	8001:8001 	  

1. 키 이름 설계

Redis에는 테이블이 없으므로 키 이름이 데이터의 의미를 전달해야 합니다.
이 강의에서는 다음 규칙을 사용합니다.

서비스:대상:식별자[:속성]

shop:product:1001
shop:product:1001:views
community:user:7:session

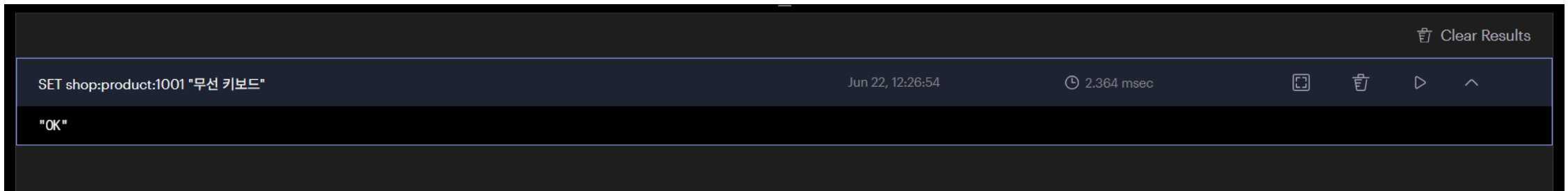
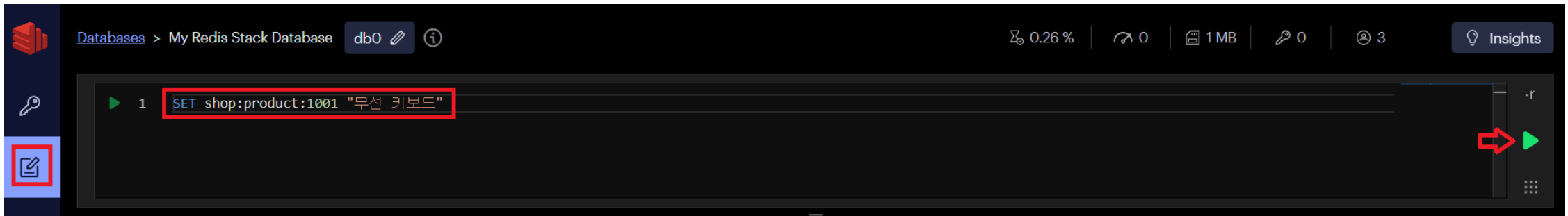
좋은 키의 조건

- 의미가 분명하고 충돌하지 않습니다.
- 너무 짧거나 지나치게 길지 않습니다.
- 같은 종류의 키는 같은 패턴을 사용합니다.
- 개인정보를 키 이름에 직접 넣지 않습니다.

2. 기본 저장과 조회

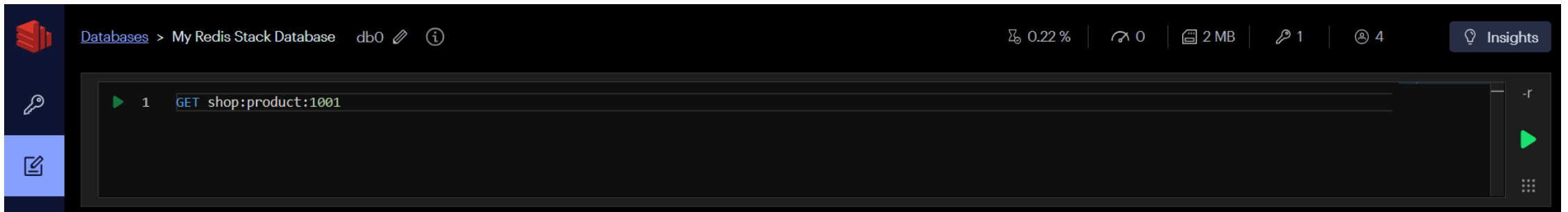
- 키(key): shop:product:1001
- 데이터(value): 무선 키보드

```
SET shop:product:1001 "무선 키보드"
```

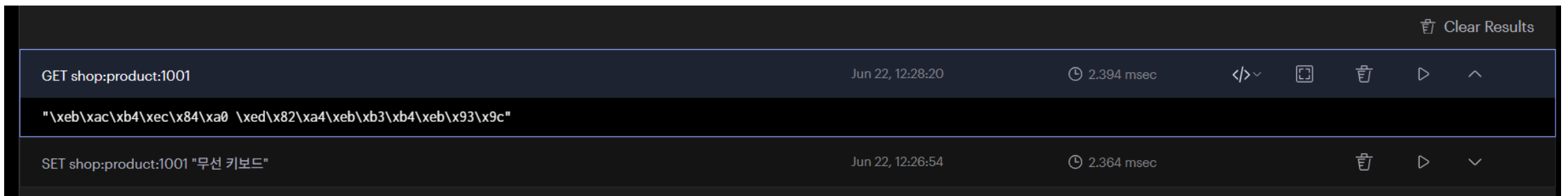


- 키(key): shop:product:1001

```
GET shop:product:1001
```



한글이 깨진 게 아니라 Redis가 문자열을 bytes escape 형태로 보여주는 것입니다.



browser에서 데이터 확인

The screenshot shows a web browser window with the address bar displaying `localhost:8001/redis-stack/browser`. The browser tab is titled "My Redis Stack Database - Browser". The interface is dark-themed and shows the Redis Stack Browser UI. On the left sidebar, the "Keys" icon (a key) is highlighted with a red square. The main content area displays a table of keys. The table has a header row with "All Key Types" and a search bar. Below the header, there is a table with one row of data. The row contains a purple "STRING" label, the key name "shop:product:1001", the value "No limit", and the size "88 B". A red arrow points to the key name "shop:product:1001". To the right of the table, there is a detailed view of the selected key. It shows the key name "shop:product:1001" in a purple box, followed by "88 B", "Length: 16", and "TTL: No limit". Below this, there is a section titled "무선 키보드" (Wireless Keyboard).

My Redis Stack Database - Browser

localhost:8001/redis-stack/browser

Databases > My Redis Stack Database db0

0.24 % | 0 | 2 MB | 1 | 4 | Insights

All Key Types Filter by Key Name or Pattern

Bulk Actions + Key

Total: 1 Last refresh: 3 min

STRING	shop:product:1001	No limit	88 B
--------	-------------------	----------	------

STRING shop:product:1001

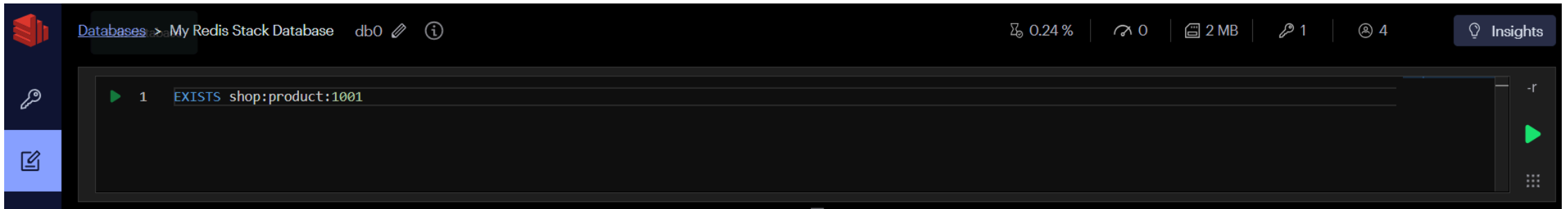
88 B Length: 16 TTL: No limit

< 1 min

무선 키보드

- EXISTS key : 키가 있으면 1, 없으면 0

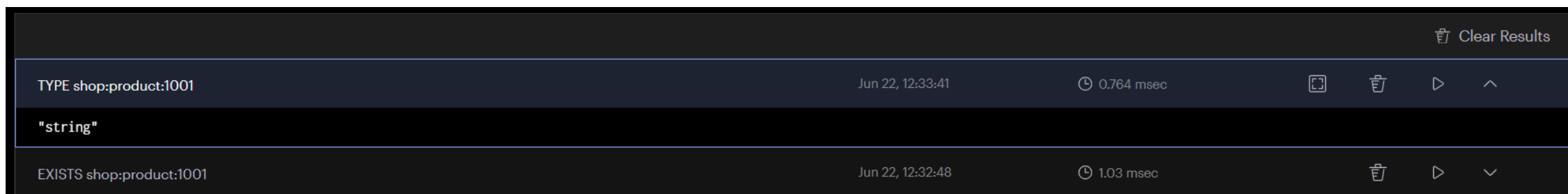
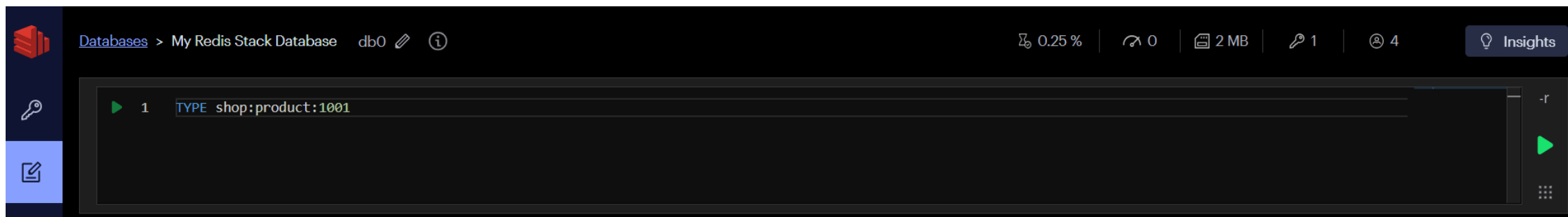
```
EXISTS shop:product:1001
```



Clear Results				
EXISTS shop:product:1001	Jun 22, 12:32:48	1.03 msec		
(integer) 1				
GET shop:product:1001	Jun 22, 12:28:20	2.394 msec		

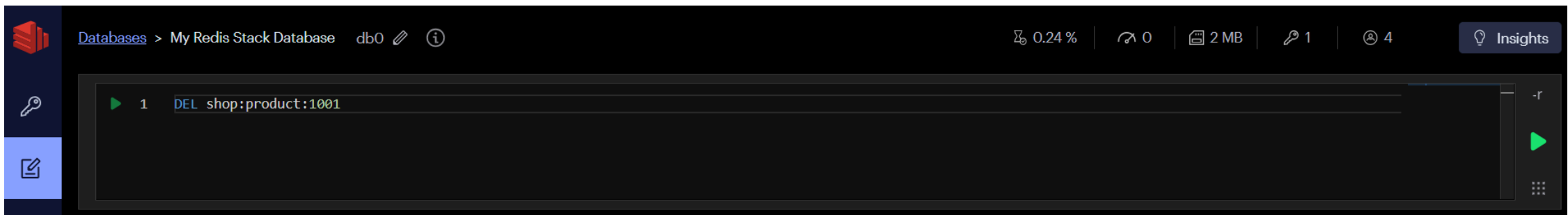
- `TYPE key` : 키에 저장된 자료형 확인

```
TYPE shop:product:1001
```



- `DEL key [key ...]` : 하나 이상의 키를 즉시 삭제

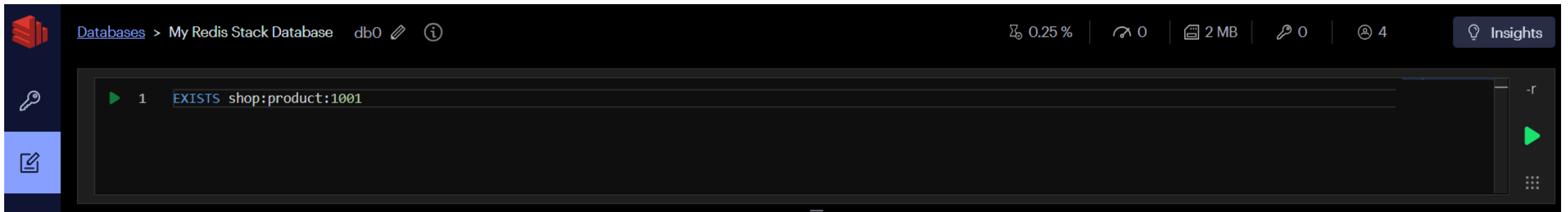
```
DEL shop:product:1001
```



Clear Results				
DEL shop:product:1001	Jun 22, 12:34:47	1.835 msec		
(integer) 1				
TYPE shop:product:1001	Jun 22, 12:33:41	0.764 msec		

- `EXISTS key` : 키가 있으면 `1`, 없으면 `0`

```
EXISTS shop:product:1001
```



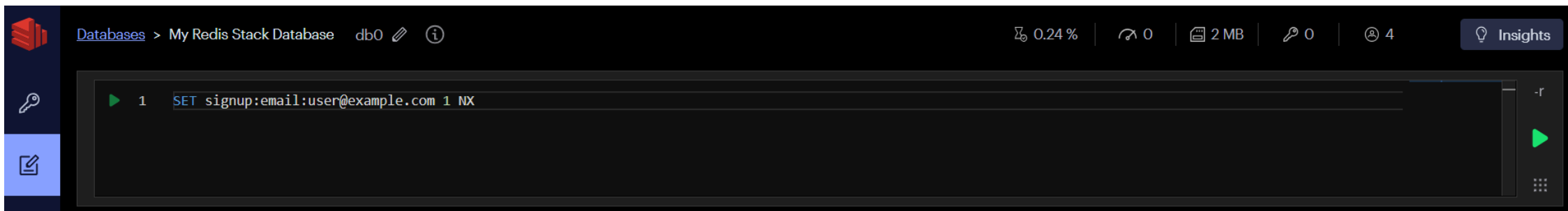
Clear Results			
EXISTS shop:product:1001	Jun 22, 12:35:37	1.665 msec	   
(integer) 0			
DEL shop:product:1001	Jun 22, 12:34:47	1.835 msec	  

3. SET 옵션

조건을 만족하지 못한 SET 은 (nil) 을 반환합니다. NX 는 간단한 중복 방지에 유용하지만, 복잡한 분산 잠금은 만료와 소유권 확인까지 별도로 설계해야 합니다.

- **NX**: 키가 없을 때만 저장

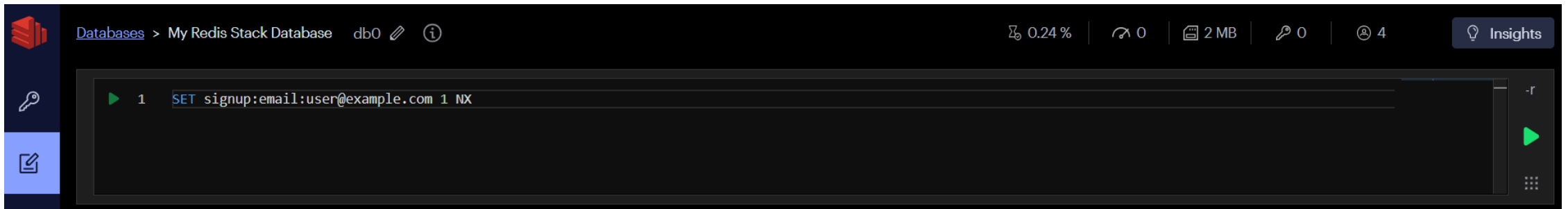
```
SET signup:email:user@example.com 1 NX
```



Clear Results			
SET signup:email:user@example.com 1 NX	Jun 22, 12:37:23	1.687 msec	   
"OK"			
EXISTS shop:product:1001	Jun 22, 12:35:37	1.665 msec	  

- **NX**: 키가 없을 때만 저장

```
SET signup:email:user@example.com 1 NX
```

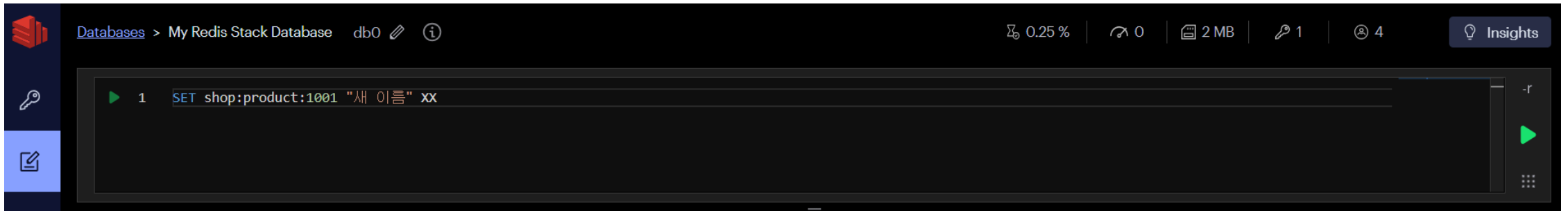


이미 저장된 키(signup:email:user@example.com)가 있기 때문에 저장 실패

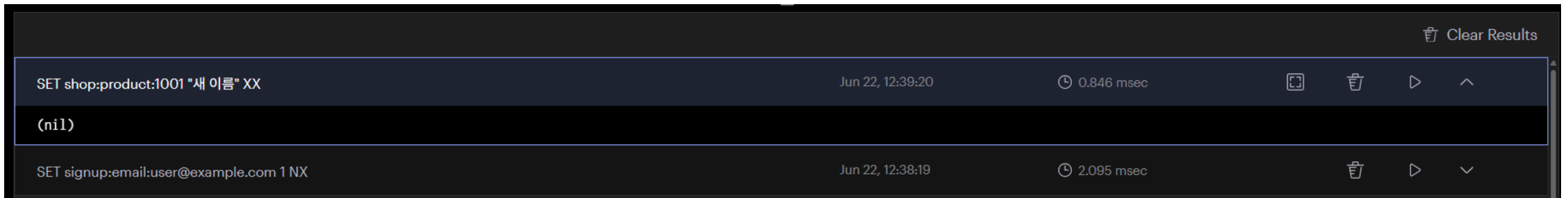
Clear Results				
SET signup:email:user@example.com 1 NX	Jun 22, 12:38:19	2.095 msec		
(nil)				
SET signup:email:user@example.com 1 NX	Jun 22, 12:37:23	1.687 msec		

- `XX`: 키가 있을 때만 저장

```
SET shop:product:1001 "새 이름" XX
```

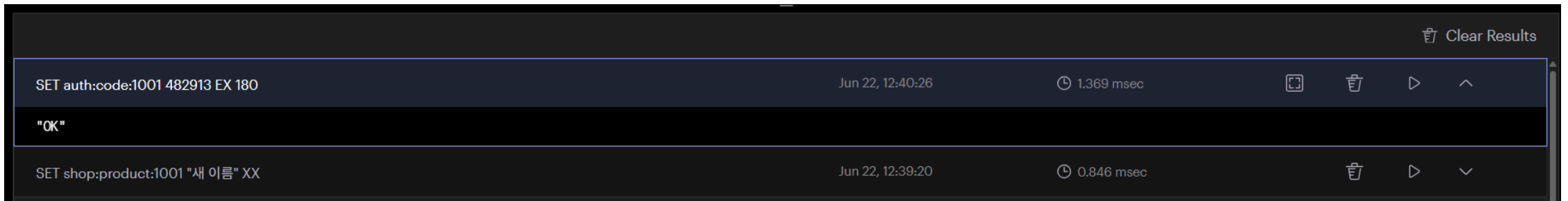
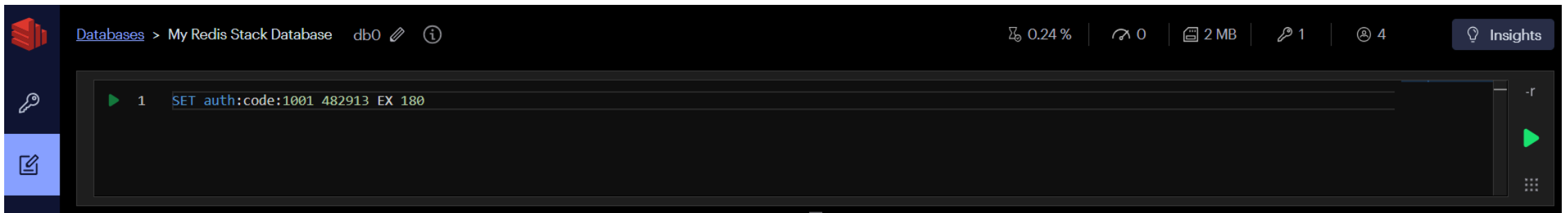


저장된 키(shop:product:1001)가 없기 때문에 저장 실패



- EX 초 : 저장과 동시에 초 단위 만료 시간 설정
- PX 밀리초 : 저장과 동시에 밀리초 단위 만료 시간 설정

```
SET auth:code:1001 482913 EX 180
```

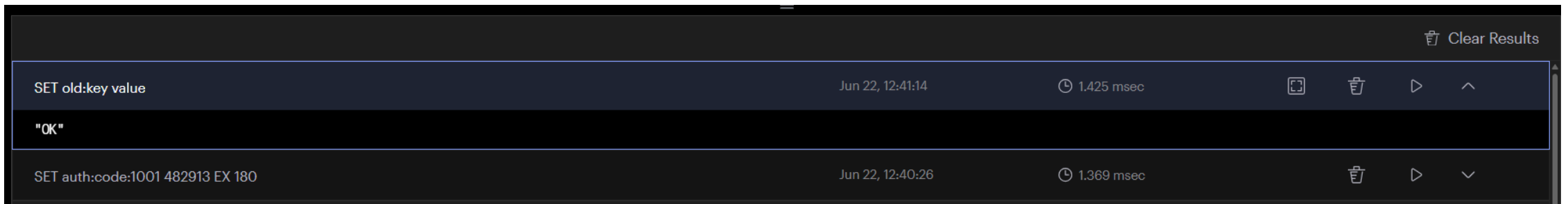
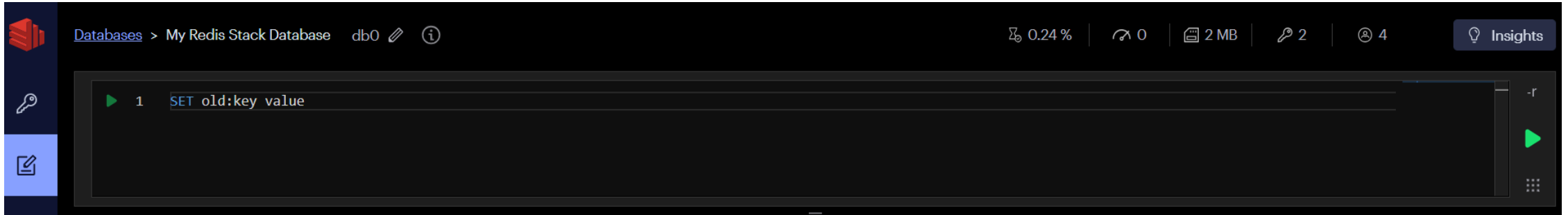


4. 키 이름 변경

`RENAME` 은 대상 키가 이미 있으면 덮어씁니다. 덮어쓰기를 막으려면 `RENAMENX` 를 사용합니다.

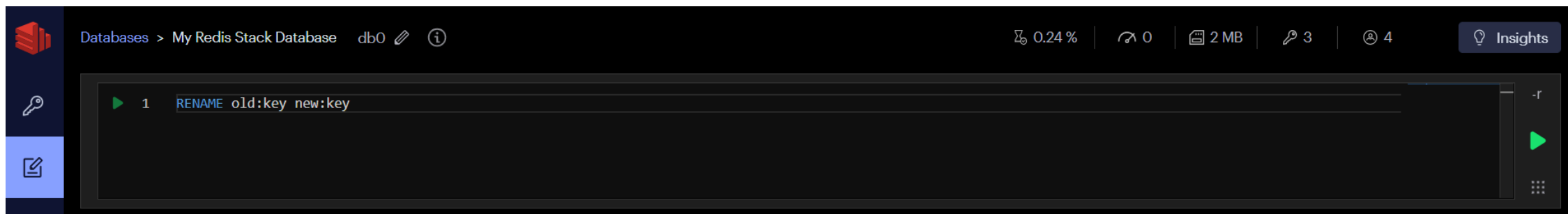
- 키 생성

```
SET old:key value
```

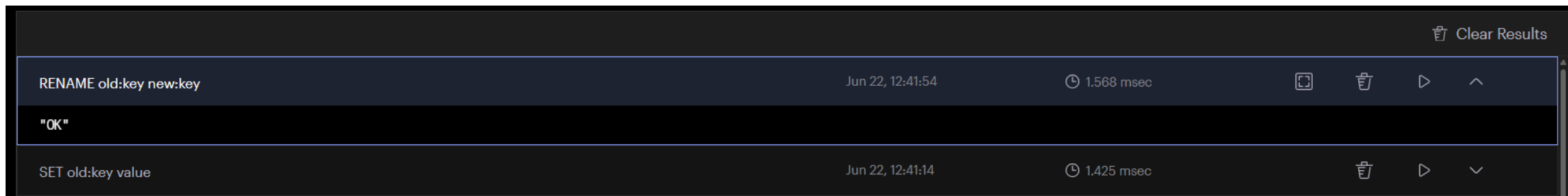


- 키 이름 변경

```
RENAME old:key new:key
```



The screenshot shows the Redis Stack database interface. The top bar indicates the database is 'My Redis Stack Database' (db0). The command 'RENAME old:key new:key' is entered in the main text area. The interface includes a sidebar with icons for databases, keys, and documents, and a top bar with various status indicators like memory usage (0.24%), connections (0), and a 'Clear Results' button.

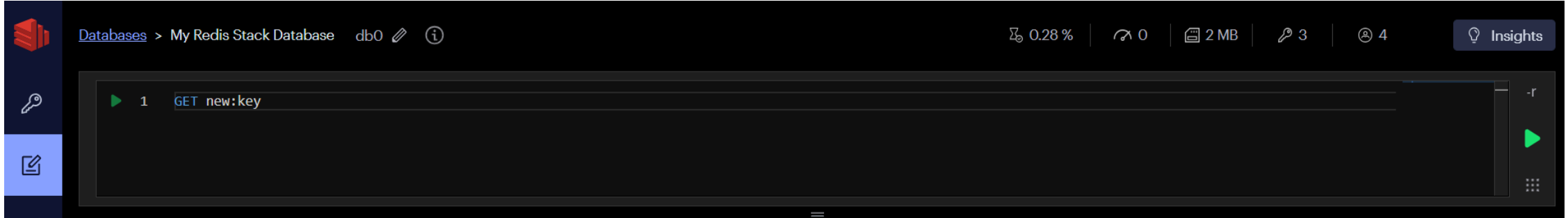


The screenshot shows the results of the RENAME command. The command 'RENAME old:key new:key' was executed on 'Jun 22, 12:41:54' and took '1.568 msec' to complete. The result is 'OK'. Below this, the previous command 'SET old:key value' is also visible, executed on 'Jun 22, 12:41:14' and taking '1.425 msec'.

Command	Timestamp	Duration	Actions
RENAME old:key new:key	Jun 22, 12:41:54	1.568 msec	Copy, Delete, Play, Up Arrow
"OK"			
SET old:key value	Jun 22, 12:41:14	1.425 msec	Copy, Play, Down Arrow

- 변경된 키 조회

GET new:key



					Clear Results
GET new:key	Jun 22, 12:42:39	2.019 msec	</>	📄	🗑️
"value"					
RENAME old:key new:key	Jun 22, 12:41:54	1.568 msec		🗑️	⏮️

5. 키 탐색: KEYS와 SCAN

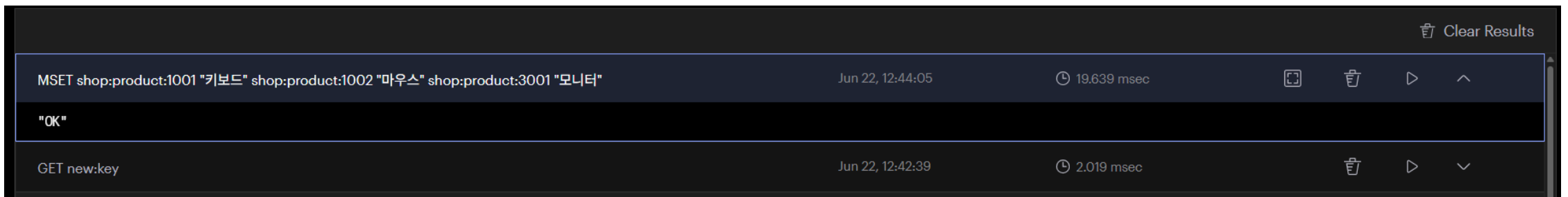
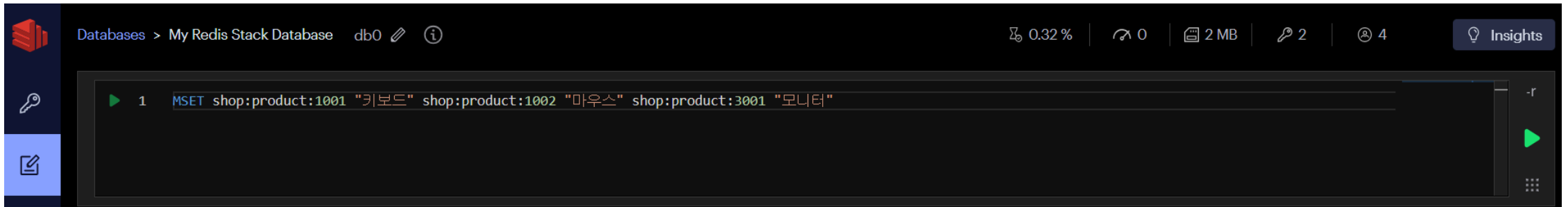
항목	KEYS	SCAN
목적	조건에 맞는 모든 Key 조회	Key를 조금씩 나누어 조회
반환 방식	한 번에 전체 반환	여러 번 호출하여 순차 반환
성능	$O(N)$ (전체 Key 탐색)	$O(1)$ (호출당 평균)
서버 영향	데이터가 많으면 Redis가 잠시 응답하지 않을 수 있음	서버 부하가 매우 적음

항목	KEYS	SCAN
운영 환경 사용	권장하지 않음	권장
개발/테스트	사용 가능	사용 가능
결과 개수	모든 Key	일부 Key (Cursor를 이용해 반복 조회)
Cursor 사용	없음	있음 (0 이 될 때까지 반복)
Pattern 검색	KEYS user:*	SCAN 0 MATCH user:*
COUNT 옵션	없음	있음 (COUNT 100 등)
대표 사용 사례	개발 중 데이터 확인	운영 서버에서 Key 조회

테스트용 데이터 생성

- **MSET** : 여러 개의 Key-Value를 한 번에 저장하는 명령어(Multiple SET)

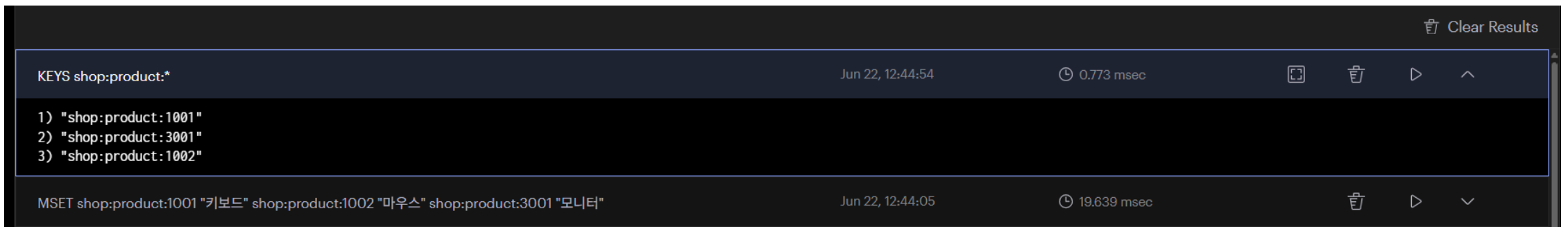
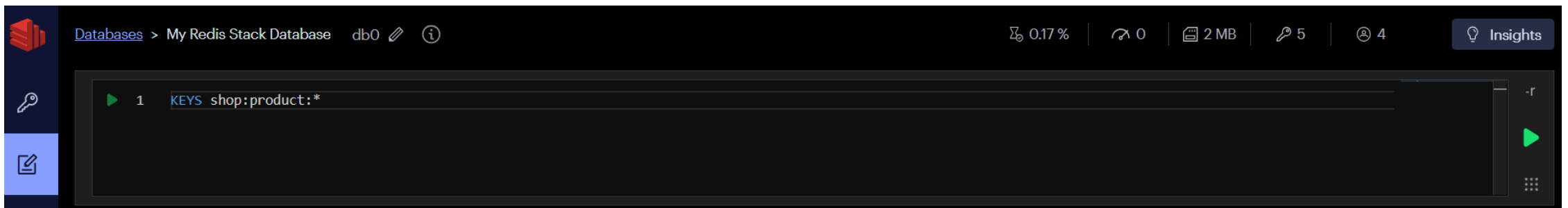
```
MSET shop:product:1001 "키보드" shop:product:1002 "마우스" shop:product:3001 "모니터"
```



KEYS

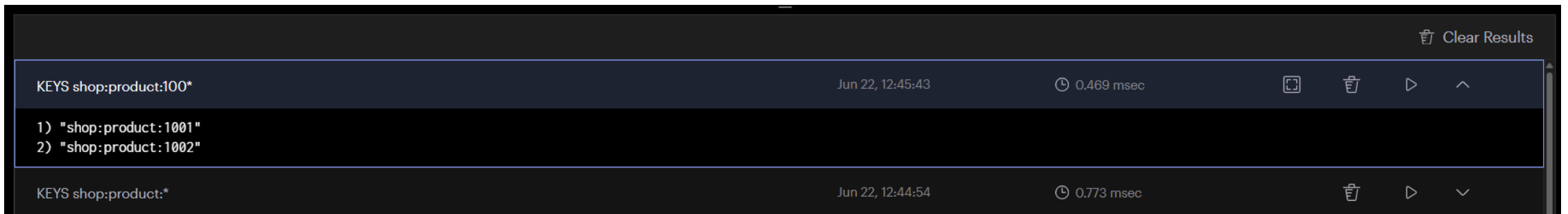
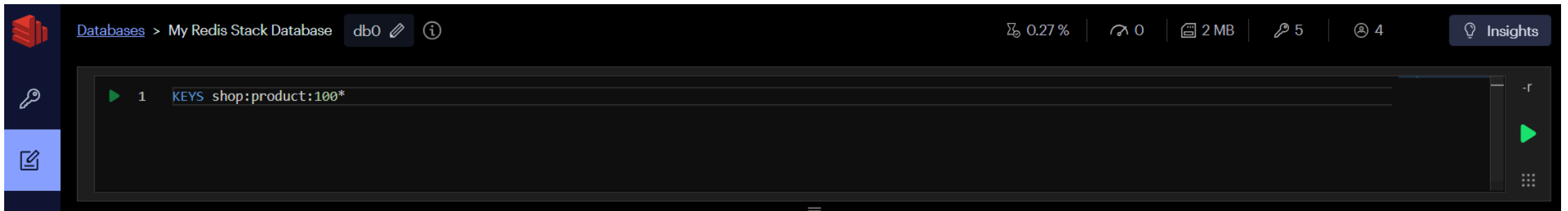
- 모든 상품 조회

```
KEYS shop:product:*
```



- 특정 상품 조회

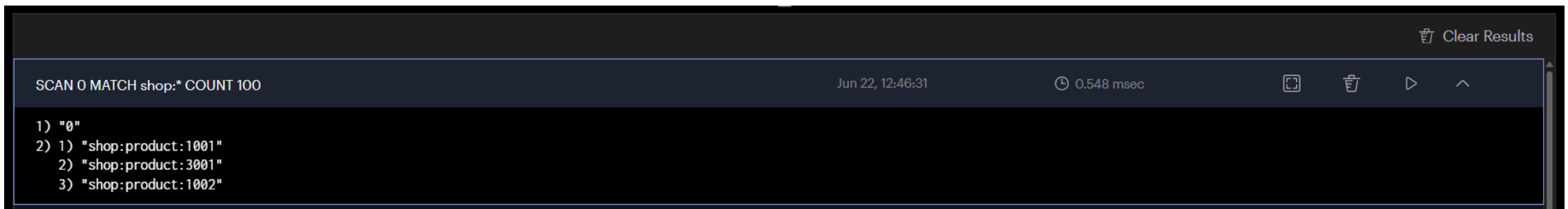
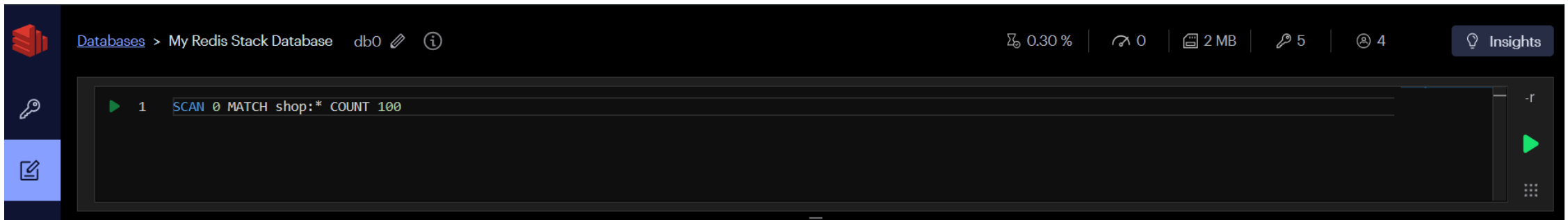
```
KEYS shop:product:100*
```



SCAN

- 모든 상품 조회

```
SCAN 0 MATCH shop:* COUNT 100
```



- 특정 상품 조회

```
SCAN 0 MATCH shop:product:100* COUNT 10
```

